Intro2cs Fall 2021-2 Moed Aleph Exam

Exam instructions:

<u>קראו בזהירות את ההנחיות!</u>

במבחן זה שני חלקים. לכל חלק מוקצות 90 דקות (לבעלי הארכת זמן תינתן תוספת בהתאם). אין לצאת לשירותים במהלך הבחינה, אלא רק בהפסקה בין שני החלקים. הבחינה תתקיים בהשגחה בזום ודרך מערכת SEB.

<u>מבנה הבחינה:</u>
בכל חלק יש

- **5 שאלות קצרות** (4 נק כל אחת). **יש לענות על <u>כל</u> השאלות הקצרות.**
- **3 שאלות תכנות** (15 נק כל אחת). **יש לענות רק על 2 מתוך ה- 3**. בסוף כל חלק תתבקשו לסמן אילו שאלות מה-3 בחרתם.

<u>הנחיות נוספות:</u>
- בשאלות התכנות עליכם לכתוב קוד יעיל ונקי ככל האפשר. תוכניות לא יעילות או שאינן כתובות היטב לא יקבלו את כל הנקודות גם אם הן עובדות נכון.
- בכל שאלות התכנות מותר לכתוב פונקציות עזר נוספות ללא הגבלה.
- אין צורך לתעד את הקוד. אין צורך ב-docstring. הערות לקוד במקומות בהם כתבתם משהו מורכב עשויות לסייע בבדיקת התשובה שלכם אך אינן הכרחיות.
- **הבחינה היא עם חומר סגור. אין לעשות שימוש במחשבון או בדפי נוסחאות, או בכל חומר עזר אחר.**
- מותר לעשות שימוש בדפי טיוטא חלקים. אין צורך להגישם.

<u>שאלות לצוות הקורס</u>
- **שאלות יש להפנות דרך הצ'אט של מודל בלבד**.
- מענה עשוי לקחת כמה דקות בגלל העומס. אם לא נעניתם אל תכתבו הודעות נוספות שיעכבו את המענה עוד יותר אלא המתינו בסבלנות.
- אנא ציינו בכל פנייה את מספר השאלה עליה אתם שואלים.
- אנא המנעו משאלות שאינן קשורות ישירות להבנת השאלה. צוות הקורס לא יוכל לספק רמזים או לענות על שאלות אחרות במהלך המבחן.
- אין צורך לענות לנו בהודעות "תודה" לאחר שהשבנו על שאלות. זה ממלא את מסכי הצ'אט ויגרום לעיכובים נוספים במענה לשאלות.

בהצלחה!

Part 1, Q1

נתונה התוכנית הבאה. הניחו כי `n` הוא חיובי ממש.

```
def deepen(lst, n):
    return [lst[:] for i in range(n)]

lst1 = list(range(n))
lst1.append(lst1)
lst2 = deepen(lst1, n)
lst3 = deepen(lst2, n)
```

ספרו את מספר הרשימות שנוצרו בתוכנית. רשימות שונות בזכרון אך עם תכולה זהה יחשבו לצורך הספירה כרשימות נפרדות (שימו לב: אם רשימה מסויימת מופיעה פעמים כ-alias אין לספור אותה שנית). כתבו את תשובותיכם לסעיפים הבאים כנוסחה התלויה ב-`n`.

חלק א: כמה רשימות שונות מכיל המשתנה `lst2` באופן ישיר או עקיף?

חלק ב: כמה רשימות שונות מכיל המשתנה `lst3` באופן ישיר או עקיף?

Part 1, Q2

נתונה הפונקציה הבאה:

```
def f(n):
    result = 0
    jump = 1
    cur = 0
    while cur < n:
        result += cur
        if jump*jump < n:
            jump *= 2
        cur += jump
    return result
```

מהי סיבוכיות זמן הריצה של `f` כפונקציה של `n` במקרה הגרוע ביותר? ציינו את החסם ההדוק ביותר.

Part 1, Q3

נתונה התוכנית הבאה:

```
def f(n, m, j):
    if n <= 0 or m <= 0:
        return 1

    if j <= 1:
        return f(n-1, m, m) + n
    else:
        return f(n, m, j-1) + m

f(n, m, m)
```

כמה פעמים נקראת הפונקציה `f` במהלך ריצת התוכנית? סיפרו גם את הקריאה ל-`f` מגוף התוכנית וגם את הקריאות הרקורסיביות.

רישמו את תשובתכם כפונקציה של המשתנים `n,m`. (הניחו שערכם של `n,m` הוא אי שלילי).

Part 1, Q4

נתונה הפונקציה הבאה:

```
def f(n):
    lst = [1,2,3]
    s = int(n**0.5)
    for i in range(n*s):
        x = [1,lst]*i
        lst.append(x)
    return lst
```

מהי **סיבוכיות המקום** של `f` כפונקציה של `n` במקרה הגרוע ביותר? כלומר, כמה זכרון תופסת הרצה של `f`? ציינו את החסם ההדוק ביותר.

Part 1, Q5

נתון קטע התוכנית הבא:

```
x = [[(i, j) for i in range(n)] for j in range(n)]
words: List[str] = ["it's", "a", "wonderful", "life"]
d = {w: (len(w), w[::-1]) for w in words}
```

חלק א: מהו הטיפוס **של המשתנה x**? ציינו את הטיפוס המדוייק ביותר שאפשר. כיתבו את תשובתכם בפורמט שיזוהה ע"י mypy.

חלק ב: מהו הטיפוס **של המשתנה d**? ציינו את הטיפוס המדוייק ביותר שאפשר. כיתבו את תשובתכם בפורמט שיזוהה ע"י mypy.

Part 1, Q6

נתונה המחלקה Node המייצגת קודקוד של עץ בינארי (אין לשנות את מימוש מחלקה זו. מותר לגשת לשדות שלה ישירות וללא getters, setters).

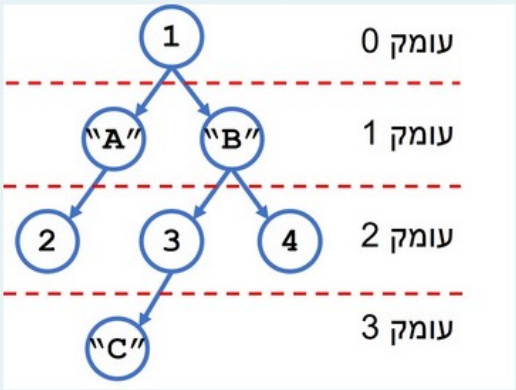
```
class Node:
    def __init__(self, data, left=None, right=None):
        self.data, self.left, self.right = data, left, right
```

עליכם לממש פונקציה `tree_iter(root, depth)` שמחזירה איטרטור שעובר על הנתונים שבעץ לפי עומק. מעומק 0 ועד עומק depth (לא כולל).

כלומר, תחילה יוחזרו הנתונים בעומק 0 של העץ (כלומר הנתון שמוחזק בשורש). לאחר מכן הנתונים בעומק 1 בעץ, אחריהם הנתונים בעומק 2 בעץ, וכן הלאה עד לעומק depth-1. אם העץ לא מגיע לעומק depth-1, האיטרטור יעצור מוקדם (כשנגמרים הנתונים להחזיר).

בכל עומק בעץ, יוחזרו האיברים לפי סדר משמאל לימין. כלומר נתונים שנמצאים בתת העץ השמאלי יוחזרו לפני אלו של תת העץ הימני (הסדר בין נתונים בתוך כל תת עץ גם הוא יהיה דומה -- קודם נתונים שמופיעים בצד שמאל ואחר כך צד ימין).

<u>לדוגמא:</u> בעץ המתואר באיור הבא



הקריאה `tree_iter(root, 3)` תחזיר איטרטור שעובר על הערכים

```
1, "A", "B", 2, 3, 4
```

לפי הסדר הנתון (משמאל לימין -- תחילה הערך `1`, אחריו `"A"` וכן הלאה).
הערך "C" לא יוחזר במסגרת האיטרטור כי הוא בעומק גדול מידי בעץ.

<u>הערות</u>:

- מותר להשתמש בגנרטור לצורך המימוש
- אין לעשות שימוש במבני נתונים אחרים כדי להחזיק את נתוני העץ או את קודקודיו
- אין לשנות את העץ בזמן הריצה
- אפשר להניח ש-root אינו None
- depth הוא מספר שלם אי שלילי
- העץ הוא עץ בינארי חוקי (הבן השמאלי והימני של כל קודקוד יהיו קודקודים חדשים או None)

Part 1, Q7

נתונה הרשימה intervals המייצגת רשימה של קטעים על הישר.
כל קטע מיוצג על ידי טאפל המכיל שני מספרים `x,y` מטיפוס `float`. כל קטע כזה הוא קטע סגור (כך ש-`x` ו- `y` עצמם נחשבים לשייכים לקטע).

הקטעים ברשימה intervals עשויים לחפוף זה לזה.

עליכם לכתוב פונקציה

```
unify(intervals: List[Tuple[float,float]]) -> List[Tuple[float,float]]
```

שתחזיר רשימה חדשה בה יופיעו הקטעים מהרשימה המקורית לאחר שמוזגו קטעים חופפים.

במילים אחרות: הקטעים ברשימה המוחזרת יהיו זרים זה לזה, ויכסו את אותם החלקים של הישר שכוסו ברשימה המקורית.

לדוגמא, הקריאה:

```
unify([(2.2, 5), (1, 4.2), (7.1,9)])
```

תחזיר

```
[(1, 5), (7.1, 9)]
```

כי זוג הקטעים הראשון חופף וימוזג בתוצאה.

<u>הערות:</u>

- הרשימה intervals יכולה להיות ריקה. מותר לשנותה בזמן הפתרון.
- כל קטע x,y נתון בצורה חוקית כך ש x קטן או שווה ל-y.
- פתרון שאינו בסיבוכיות זמן יעילה מספיק לא יקבל את מירב הנקודות.

Part 1, Q8

נזכור שב**עץ חיפוש בינארי** המכיל מספרים מתקיימת התכונה הבאה: לכל קודקוד v, המספר שבקודקוד עצמו גדול או שווה מכל מספר בתת העץ השמאלי של v, וקטן ממש מכל מספר בתת העץ הימני של v.

כתבו פונקציה

```
is_legal_BST(root : Node) -> bool
```

המקבלת שורש של עץ בינארי ובודקת אם העץ הנתון הוא עץ חיפוש בינארי חוקי (a legal Binary Search Tree). הפונקציה מחזירה `True` אם העץ חוקי, ו-`False` אחרת.

העץ הבינארי נתון ע"י המחלקה `Node`. אין לשנותה. מותר לגשת ישירות לשדות שלה, ואין צורך במתודות `get` או `set`.

<u>הנחות על הקלט:</u>

- `root` הוא מטיפוס `Node` ובפרט אינו `None`. **מבנה העץ תקין (אין מעגלים וכל ההפניות הן לאובייקטים מטיפוס `Node`, או ל-`None` כמקובל בעצים)**
- העץ מכיל נתונים מספריים בלבד (מטיפוס `float` או `int`)

<u>לדוגמא:</u>

העץ הבא <u>אינו</u> חוקי: `Node(3, Node(8), Node(7))`

כי בשורש יש את הערך 3 אך בתת העץ השמאלי שלו יש מספר שגדול מהשורש (8).

ואילו העץ הבא הוא חוקי: `Node(3, Node(2), Node(7))`

Part 2, Q1

נתונה רשימה `lst`. יהי `n` אורך הרשימה.

```
def my_add(lst, f, t):
    if f == t-1:
        return lst[f]
    if f >= t:
        return 0
    mid = (f+t)//2
    return my_add(lst, f, mid) + my_add(lst, mid, t)

n = len(lst)
my_add(lst, 0, n)
```

מהי סיבוכיות זמן הריצה של התוכנית כפונקציה של `n` במקרה הגרוע ביותר? ציינו את החסם ההדוק ביותר.

Part 2, Q2

שימו לב! הבהרה לשאלה: בקטעי הקוד שהושמטו אין לולאות אינסופיות, ואין דריסה של הפונקציה print או שימוש כלשהוא ב-import.

נתונה פונקציה ממנה הושמטו חלקים מהקוד. החלקים בהם הושמט קוד מסומנים כ-

`<CODE1>, <CODE2>,<CODE3>`

ידוע כי:

- מקטעי הקוד החסרים לא כללו הדפסות.
- רק הקוד ב- `CODE2` עשוי היה לייצר Exception מסוג כלשהוא (אך לא תמיד). מקטעי הקוד האחרים לעולם לא זורקים Exceptions.
- מלבד זאת, מקטעי הקוד יכלו לכלול כל פקודה אחרת בפייתון כולל קריאה לפונקציות, `return`, תנאים, לולאות, וכד'.
- כל מקטע קוד יכול היה להכיל יותר מפקודה יחידה.

```
def f(x):
    try:
        print("A", end="")
        <CODE1>
        print("B", end="")
        <CODE2> #may raise an exception
        print("C", end="")
        <CODE3>
        print("D", end="")
    except:
        print("E", end="")

    finally:
        print("F", end="")
```

אילו מהמחרוזות הבאות יכולות להיות מודפסות כתוצאה מריצתה של הפונקציה המקורית (לפני מחיקת מקטעי הקוד)?
סמנו את כל האפשרויות הנכונות.

- ☐ AF
- ☐ ABCD
- ☐ ABEF
- ☐ ABCDEF

Part 2, Q3

נתון קטע התוכנית הבא:

```
thingymajingy = ?

@thingymajingy(117)
def foo(x, y, z):
    return x+y+z

@thingymajingy(321)
def boo(u, v):
    return u > v+3

foo()   # returns 117
boo()   # returns 321
```

מה צריך לרשום במקום סימן השאלה כדי שהקריאה ל-`foo` תחזיר 117, והקריאה ל `boo` תחזיר 321?

Part 2, Q4

נתונה התוכנית הבאה. נזכור שהפונקציה `after` מוסיפה מאורע לתור המאורעות של ה-GUI.

```
import tkinter

top = tkinter.Tk()
label = tkinter.Label(top, height=20, width=20)
label.pack()
label["bg"] = "green"

def f(col):
    def inner():
        label["bg"] = col
    return inner

def g():
    label.after(0, f("blue"))
    label.after(2000, g)
    label.after(1000, f(label["bg"]))

top.after(2500, f("red"))
top.after(1000, g)
top.mainloop()
```

באילו צבעים תיצבע התווית (label)? ציינו את הצבעים השונים לפי הסדר (אם הצבע אינו משתנה החל מרגע מסויים, ציינו שוב את אותו צבע).

הצבע הראשון:

הצבע השני:

הצבע השלישי:

הצבע הרביעי:

הצבע החמישי:

הצבע השישי:

הצבע השביעי:

Part 2, Q5

נתונה המחלקה הבאה המייצגת קודקוד בעץ. למחלקה יש מתודה `f` אותה עליכם להשלים בשתי דרכים שונות.

במחלקה `Node`, השדה `children` מכיל רשימה של קודקודים שהם ילדיו של הקודקוד הנוכחי (יכול להיות מספר כלשהוא של ילדים לכל קודקוד כולל גם 0 ילדים לעלים).

```
class Node():
    def __init__(self, data, children):
        self.data, self.children = data, children

    def f(self):
        down = [child.f() for child in self.children]
        return ?
```

חלק א: מה צריך לרשום בתוך המתודה f במקום סימן השאלה על מנת שהיא תחזיר את **מספר הקודקודים שבעץ**?

חלק ב: הניחו שהנתונים בעץ (בשדה data) ניתנים להשוואה. מה צריך לרשום בתוך המתודה f במקום סימן השאלה על מנת שהיא תחזיר את **הנתון המקסימלי שבעץ**?

בשני הסעיפים כתבו רק את מה שצריך לכתוב במקום סימן השאלה ואל תכללו את המילה `return`.

Part 2, Q6

שימו לב!

1) בשאלה זו אסור לעשות שימוש בכלים מitertools ובפרט אסור לעשות שימוש בפונקציות של פייתון שעוברות על כל הצירופים (כגון combinations).

2) כל משימה מוקצית לגמד יחיד. אין שני גמדים (או יותר) שעובדים יחד על אותה משימה.

במפעל השוקולד של ווילי וונקה עובדים הרבה גמדים קטנים. לעובדים יש משימות שעליהם להשלים במהרה. כל משימה מיוצגת ע"י מספר (חיובי) יחיד מטיפוס `float` המייצג את מספר השעות שלוקח להשלימה. כל המשימות נתונות יחד ברשימה `tasks` (זו רשימה של מספרים מטיפוס `float`).

ווילי רוצה לחלק את המשימות לעובדים בצורה מיטבית. הוא רוצה למצוא חלוקה של המשימות לעובדיו שתביא למינימום את זמן הסיום של המשימות כולן (כלומר להקדים ככל הניתן את זמן הסיום של המשימה האחרונה שמסתיימת).

הגמדים החרוצים עובדים במקביל זה לזה על משימותיהם השונות. זמן העבודה שדרוש לכל גמד כדי לסיים את משימותיו הוא סכום הזמנים של המשימות שהוקצו לו.

כתבו פונקציה

```
min_time(num_workers: int, tasks: List[float])  -> float
```

המקבלת את מספר העובדים במפעל של ווילי ואת רשימת המשימות ומחזירה את זמן הסיום הטוב ביותר שניתן להשיג.

<u>לדוגמא:</u>

אם ישנם 4 גמדים שעובדים במפעל, ורשימת המשימות היא [4 ,2.5 ,2 ,5 ,3 ,2] זמן הסיום הקצר ביותר יהיה **5**.

זאת כי ניתן לתת לגמד הראשון את המשימה באורך 5, לגמד השני את המשימות באורך [3,2], לגמד השלישי משימות באורך [2,2.5] ולגמד הרביעי משימה באורך 4 (שימו לב ששני הגמדים הראשונים בחלוקה זו מסיימים את המשימה האחרונה שלהם באותו רגע -- כ-5 שעות לאחר שהחלו ואילו כל שאר הגמדים מסיימים קודם). כל השמה אחרת תיתן זמן סיום דומה או גרוע יותר.

<u>הערות:</u>

לצערנו לא ידועה דרך יעילה מאוד לפתרון הבעיה ויש לעבור על אפשרויות רבות לחלוקה של המשימות בין העובדים. עשו שימוש ב-backtracking.

<u>הנחות על הקלט:</u>

- יש לפחות עובד אחד.
- רשימת המשימות יכולה להיות רשימה ריקה (ואז זמן הסיום הוא 0).
- מותר להניח שרשימת המשימות מכילה רק ערכים מטיפוס `float`. כולם חיוביים ממש.

Part 2, Q7

ממשו מחלקה בשם `Cycle` המתארת "מעגל". כלומר, מבנה נתונים שמחזיק נתונים לפי סדר מעגלי. בכל רגע, המעגל מצביע על אחד מהנתונים ומאפשר לבצע פעולות שקשורות אליו. ניתן להתקדם לנתון הבא בעזרת פעולת `rotate` ולחזור לנתון הקודם בעזרת `rotate_back`. פעולות אלו הן מעגליות. כלומר, אם האובייקט כרגע נמצא בנתון האחרון, פעולת `rotate` תעבור לנתון הראשון. באופן דומה פעולת `rotate_back` מהנתון הראשון תחזור לנתון האחרון.

שימו לב: כל אחת מהפעולות במחלקה Cycle צריכה להיות יעילה ככל הניתן!

הפעולות הנתמכות:

- `c = Cycle(data)`

מתודה זו תייצר מעגל שמחזיק נתון יחיד שהוא data.

- `c.insert_next(data)`

מתודה זו תכניס את data לתוך מבנה הנתונים כנתון הבא (אך לא תעבור אליו). המתודה אינה מחזירה דבר.

- `c.rotate()`

מתודה זו עוברת לנתון הבא ומחזירה את הנתון אליו עברנו. המעבר מתבצע באופן מעגלי (מהנתון האחרון חוזרים לראשון).

- `c.rotate_back()`

המתודה עוברת לנתון הקודם ומחזירה את הנתון אליו עברנו. המעבר מתבצע באופן מעגלי (מהנתון הראשון נחזור לאחרון).

- `c.delete()`

מתודה זו מסירה את הנתון הנוכחי מהמעגל. היא אינה מחזירה דבר. לאחר ההסרה, מתקדמים לנתון הבא (שאחרי זה שהוסר). אם יש נתון יחיד במעגל, ההסרה נכשלת ונזרקת שגיאה מסוג `LookupError`. (שגיאה זו כבר מוגדרת בפייתון).

<u>לדוגמא:</u>

```
c = Cycle(3)            #c: [3]
c.insert_next(2)        #c: [3,2]
c.insert_next(1)        #c: [3,1,2]
print(c.rotate())       #c: [3,1,2] prints: 1
print(c.rotate())       #c: [3,1,2] prints: 2
print(c.rotate())       #c: [3,1,2] prints: 3
print(c.rotate())       #c: [3,1,2] prints: 1
c.delete()              #c: [3,2]
print(c.rotate_back()) #c: [3,2] prints: 3
```

*בדוגמא הנ"ל הנתון "הנוכחי" בכל שלב מודגש ומסומן בקו תחתון.

<u>הערות:</u>

- מותר להוסיף שיטות נוספות ומחלקות נוספות לפי הצורך.
- בשאלה זו היעילות חשובה. פתרון בסיבוכיות זמן ריצה לא אופטימלית לא יקבל את מלוא הנקודות.

Part 2, Q8

נתון מילון `d` הממפה מספרים שלמים אי שליליים (מטיפוס `int`) לערכים כלשהם.

כתבו פונקציה בשם `fix` כך ש `fix(d)` תשמש כ-decorator לפונקציות ותקבע מראש את ערכם של חלק מהפרמטרים של הפונקציה שקושטה.

כך, לאחר הקישוט של פונקציה `f` כלשהיא בעזרת `fix(d)`, הפונקציה החדשה תקבל פחות ערכים מההגדרה המקורית של `f`. הערכים החסרים ילקחו מהמילון `d`.

אם המילון מיפה את המפתח `i` לערך `val`, אזי הקריאה ל-`f` המקושטת תמיד תיתן את הערך `val` לפרמטר ה-`i` של הפונקציה (ראו דוגמאות בהמשך).
מלבד זאת, הפונקציה המקושטת תתנהג באופן זהה לחלוטין למקורית (למשל יהיה לה אותו ערך החזרה וכד').

<u>לדוגמא:</u>
אם לפונקציה `f` היו במקור 4 פרמטרים `x,y,z,w`, ואם המילון `d` מיפה את המפתח 0 אל הערך `"hello"` ואת המפתח 2 לערך `"goodbye"`, אזי קריאה לפונקציה המקושטת ע"י `fix(d)` באופן הבא: `f(u,v)` תבצע למעשה את אותו חישוב שמבצעת הקריאה

```
f("hello", u, "goodbye", v)
```

בפונקציה המקורית. כלומר הערכים במקומות 0 ו-2 נקבעים כ- `"hello"` ו- `"goodbye"` בהתאמה, ושאר הפרמטרים נלקחים מהקריאה לפונקציה המקושטת לפי הסדר.

<u>דוגמא נוספת:</u>
התוכנית הבאה רצה ללא שגיאות.

```
@fix({0:"a", 2:"c"})
def g(x,y,z):
    return x+y+z

assert g("b") == "abc"
```

<u>הנחות על הקלט:</u>

- הפונקציה אותה מקשטים היא בעלת 0 ארגומנטים או יותר.
- הפונקציה שמקשטים לא עושה שימוש ב-keyword arguments כלל.
- המילון ש-fix מקבלת מכיל מפתחות וערכים חוקיים (שהפונקציה המקורית יכולה לקבל). מספר המפתחות במילון הוא לכל היותר מספר הפרמטרים של הפונקציה אותה מקשטים. המילון יכול להיות ריק.
- אסור לעשות שימוש בפונקציות מ-functools לצורך הפתרון (ואין צורך להפעיל את wraps עבור הדקורטור) .